4 Software processes

Objectives

The objective of this chapter is to introduce you to the idea of a software process—a coherent set of activities for software production. When you have read this chapter, you will:

- understand the concept of software processes and software process models;
- understand three generic software process models and when they might be used;
- understand, in outline, the activities involved in software requirements engineering, software development, testing and evolution;
- understand how the Rational Unified Process integrates good software process practice to create a modern, generic process model;
- have been introduced to CASE technology that is used to support software process activities.

Contents

4.1 Software process models
4.2 Process iteration
4.3 Process activities
4.4 The Rational Unified Process
4.5 Computer-Aided Software Engineering

A software process is a set of activities that leads to the production of a software product. These activities may involve the development of software from scratch in a standard programming language like Java or C. Increasingly, however, new software is developed by extending and modifying existing systems and by configuring and integrating off-the-shelf software or system components.

Software processes are complex and, like all intellectual and creative processes, rely on people making decisions and judgements. Because of the need for judgement and creativity, attempts to automate software processes have met with limited success. Computer-aided software engineering (CASE) tools (discussed in Section 4.5) can support some process activities. However, there is no possibility, at least in the next few years, of more extensive automation where software takes over creative design from the engineers involved in the software process.

One reason the effectiveness of CASE tools is limited is because of the immense diversity of software processes. There is no ideal process, and many organisations have developed their own approach to software development. Processes have evolved to exploit the capabilities of the people in an organisation and the specific characteristics of the systems that are being developed. For some systems, such as critical systems, a very structured development process is required. For business systems, with rapidly changing requirements, a flexible, agile process is likely to be more effective.

Although there are many software processes, some fundamental activities are common to all software processes:

1. *Software specification* The functionality of the software and constraints on its operation must be defined.
2. *Software design and implementation* The software to meet the specification must be produced.
3. *Software validation* The software must be validated to ensure that it does what the customer wants.
4. *Software evolution* The software must evolve to meet changing customer needs.

I discuss these activities briefly in this chapter and discuss them in much more detail in later parts of the book.

Although there is no 'ideal' software process, there is scope for improving the software process in many organisations. Processes may include outdated techniques or may not take advantage of the best practice in industrial software engineering. Indeed, many organisations still do not take advantage of software engineering methods in their software development.

Software processes can be improved by process standardisation where the diversity in software processes across an organisation is reduced. This leads to improved communication and a reduction in training time, and makes automated process support more economical. Standardisation is also an important first step in introducing

new software engineering methods and techniques and good software engineering practice. I discuss software process improvement in more detail in Chapter 28.

4.1 Software process models

As I explained in Chapter 1, a software process model is an abstract representation of a software process. Each process model represents a process from a particular perspective, and thus provides only partial information about that process. In this section, I introduce a number of very general process models (sometimes called *process paradigms*) and present these from an architectural perspective. That is, we see the framework of the process but not the details of specific activities.

These generic models are not definitive descriptions of software processes. Rather, they are abstractions of the process that can be used to explain different approaches to software development. You can think of them as process frameworks that may be extended and adapted to create more specific software engineering processes.

The process models that I cover here are:

1. *The waterfall model* This takes the fundamental process activities of specification, development, validation and evolution and represents them as separate process phases such as requirements specification, software design, implementation, testing and so on.
2. *Evolutionary development* This approach interleaves the activities of specification, development and validation. An initial system is rapidly developed from abstract specifications. This is then refined with customer input to produce a system that satisfies the customer s needs.
3. *Component-based software engineering* This approach is based on the existence of a significant number of reusable components. The system development process focuses on integrating these components into a system rather than developing them from scratch.

These three generic process models are widely used in current software engineering practice. They are not mutually exclusive and are often used together, especially for large systems development. Indeed, the Rational Unified Process that I cover in Section 4.4 combines elements of all of these models. Sub-systems within a larger system may be developed using different approaches. Therefore, although it is convenient to discuss these models separately, you should understand that, in practice, they are often combined.

All sorts of variants of these generic processes have been proposed and may be used in some organisations. The most important variant is probably formal system development, where a formal mathematical model of a system is created. This model

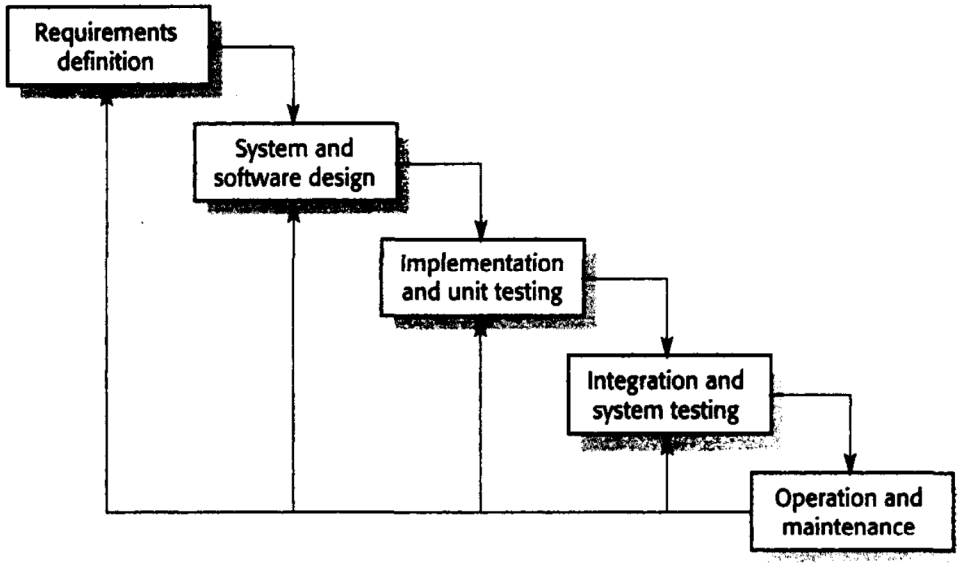


Figure 4.1 The software life cycle

is then transformed, using mathematical transformations that preserve its consistency, into executable code.

The best-known example of a formal development process is the Cleanroom process, which was originally developed by IBM (Mills, et al., 1987; Selby, et al., 1987; Linger, 1994; Prowell, et al., 1999). In the Cleanroom process each software increment is formally specified and this specification is transformed into an implementation. Software correctness is demonstrated using a formal approach. There is no testing for defects in the process, and the system testing is focused on assessing the system's reliability.

Both the Cleanroom approach and another approach to formal development based on the B method (Wordsworth, 1996) are particularly suited to the development of systems that have stringent safety, reliability or security requirements. The formal approach simplifies the production of a safety or security case that demonstrates to customers or certification bodies that the system does actually meet the safety or security requirements.

Outside of these specialised domains, processes based on formal transformations are not widely used. They require specialised expertise and, in reality, for the majority of systems this process does not offer significant cost or quality advantages over other approaches to system development.

4.1.1 The waterfall model

The first published model of the software development process was derived from more general system engineering processes (Royce, 1970). This is illustrated in Figure 4.1. Because of the cascade from one phase to another, this model is known as the waterfall model or software life cycle. The principal stages of the model map onto fundamental development activities:

1. *Requirements analysis and definition* The system's services, constraints and goals are established by consultation with system users. They are then defined in detail and serve as a system specification.

2. *System and software design* The systems design process partitions the requirements to either hardware or software systems. It establishes an overall system architecture. Software design involves identifying and describing the fundamental software system abstractions and their relationships.

3. *Implementation and unit testing* During this stage, the software design is realised as a set of programs or program units. Unit testing involves verifying that each unit meets its specification.

4. *Integration and system testing* The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met. After testing, the software system is delivered to the customer.

5. *Operation and maintenance* Normally (although not necessarily) this is the longest life-cycle phase. The system is installed and put into practical use. Maintenance involves correcting errors which were not discovered in earlier stages of the life cycle, improving the implementation of system units and enhancing the system's services as new requirements are discovered.

In principle, the result of each phase is one or more documents that are approved ('signed off'). The following phase should not start until the previous phase has finished. In practice, these stages overlap and feed information to each other. During design, problems with requirements are identified; during coding design problems are found and so on. The software process is not a simple linear model but involves a sequence of iterations of the development activities.

Because of the costs of producing and approving documents, iterations are costly and involve significant rework. Therefore, after a small number of iterations, it is normal to freeze parts of the development, such as the specification, and to continue with the later development stages. Problems are left for later resolution, ignored or programmed around. This premature freezing of requirements may mean that the system won't do what the user wants. It may also lead to badly structured systems as design problems are circumvented by implementation tricks.

During the final life-cycle phase (operation and maintenance), the software is put into use. Errors and omissions in the original software requirements are discovered. Program and design errors emerge and the need for new functionality is identified. The system must therefore evolve to remain useful. Making these changes (software maintenance) may involve repeating previous process stages.

The advantages of the waterfall model are that documentation is produced at each phase and that it fits with other engineering process models. Its major problem is its inflexible partitioning of the project into distinct stages. Commitments must be made at an early stage in the process, which makes it difficult to respond to changing customer requirements.

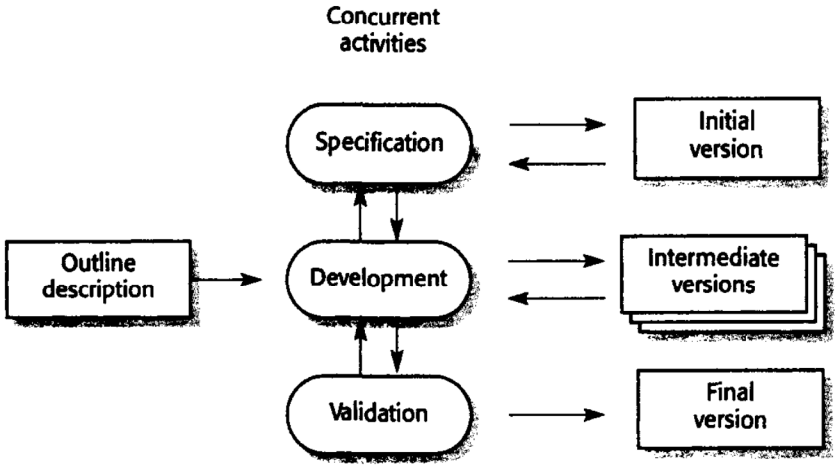


Figure 4.2 Evolutionary development

Therefore, the waterfall model should only be used when the requirements are well understood and unlikely to change radically during system development. However, the waterfall model reflects the type of process model used in other engineering projects. Consequently, software processes based on this approach are still used for software development, particularly when the software project is part of a larger systems engineering project.

4.1.2 Evolutionary development

Evolutionary development is based on the idea of developing an initial implementation, exposing this to user comment and refining it through many versions until an adequate system has been developed (Figure 4.2). Specification, development and validation activities are interleaved rather than separate, with rapid feedback across activities.

There are two fundamental types of evolutionary development:

1. *Exploratory development* where the objective of the process is to work with the customer to explore their requirements and deliver a final system. The development starts with the parts of the system that are understood. The system evolves by adding new features proposed by the customer.
2. *Throwaway prototyping* where the objective of the evolutionary development process is to understand the customer's requirements and hence develop a better requirements definition for the system. The prototype concentrates on experimenting with the customer requirements that are poorly understood.

An evolutionary approach to software development is often more effective than the waterfall approach in producing systems that meet the immediate needs of customers. The advantage of a software process that is based on an evolutionary approach

is that the specification can be developed incrementally. As users develop a better understanding of their problem, this can be reflected in the software system. However, from an engineering and management perspective, the evolutionary approach has two problems:

1. *The process is not visible* Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
2. *Systems are often poorly structured* Continual change tends to corrupt the software structure. Incorporating software changes becomes increasingly difficult and costly.

For small and medium-sized systems (up to 500,000 lines of code), I think that the evolutionary approach is the best approach to development. The problems of evolutionary development become particularly acute for large, complex, long-lifetime systems, where different teams develop different parts of the system. It is difficult to establish a stable system architecture using this approach, which makes it hard to integrate contributions from the teams.

For large systems, I recommend a mixed process that incorporates the best features of the waterfall and the evolutionary development models. This may involve developing a throwaway prototype using an evolutionary approach to resolve uncertainties in the system specification. You can then reimplement the system using a more structured approach. Parts of the system that are well understood can be specified and developed using a waterfall-based process. Other parts of the system, such as the user interface, which are difficult to specify in advance, should always be developed using an exploratory programming approach.

Evolutionary development processes and process support are covered in more detail in Chapter 17, along with system prototyping and agile software development. Evolutionary development is also incorporated in the Rational Unified Process that I discuss later in this chapter.

Component based software engineering

In the majority of software projects, there is some software reuse. This usually happens informally when people working on the project know of designs or code which is similar to that required. They look for these, modify them as needed and incorporate them into their system. In the evolutionary approach, described in Section 4.1.2, reuse is often essential for rapid system development.

This informal reuse takes place irrespective of the development process that is used. However, in the last few years, an approach to software development called component-based software engineering (CBSE), which relies on reuse, has emerged and is becoming increasingly used. I briefly introduce this approach here but cover it in more detail in Chapter 19.

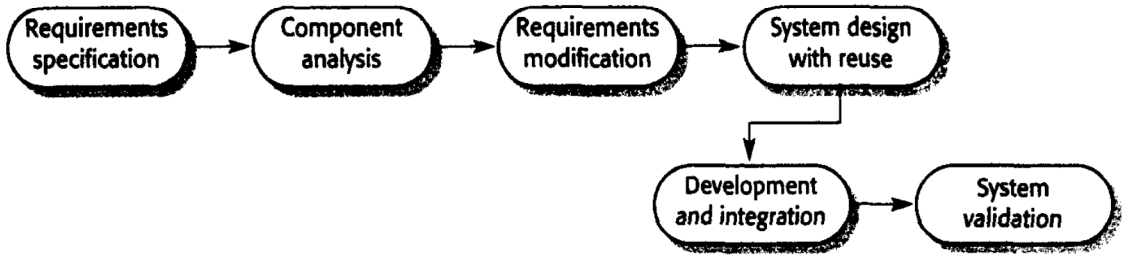


Figure 4.3 Component-based software engineering

This reuse-oriented approach relies on a large base of reusable software components and some integrating framework for these components. Sometimes, these components are systems in their own right (COTS or commercial off-the-shelf systems) that may provide specific functionality such as text formatting or numeric calculation. The generic process model for CBSE is shown in Figure 4.3.

While the initial requirements specification stage and the validation stage are comparable with other processes, the intermediate stages in a reuse-oriented process are different. These stages are:

1. *Component analysis* Given the requirements specification, a search is made for components to implement that specification. Usually, there is no exact match, and the components that may be used only provide some of the functionality required.

2. *Requirements modification* During this stage, the requirements are analysed using information about the components that have been discovered. They are then modified to reflect the available components. Where modifications are impossible, the component analysis activity may be re-entered to search for alternative solutions.

3. *System design with reuse* During this phase, the framework of the system is designed or an existing framework is reused. The designers take into account the components that are reused and organise the framework to cater to this. Some new software may have to be designed if reusable components are not available.

4. *Development and integration* Software that cannot be externally procured is developed, and the components and COTS systems are integrated to create the new system. System integration, in this model, may be part of the development process rather than a separate activity.

Component-based software engineering has the obvious advantage of reducing the amount of software to be developed and so reducing cost and risks. It usually also leads to faster delivery of the software. However, requirements compromises are inevitable and this may lead to a system that does not meet the real needs of users. Furthermore, some control over the system evolution is lost as new versions of the reusable components are not under the control of the organisation using them.

CBSE has much in common with an emerging approach to system development that is based on integrating web services from a range of suppliers. I cover this service-centric development approach in Chapter 12.

4.2 Process iteration

Change is inevitable in all large software projects. The system requirements change as the business procuring the system responds to external pressures. Management priorities change. As new technologies become available, designs and implementation change. This means that the software process is not a one-off process; rather, the process activities are regularly repeated as the system is reworked in response to change requests.

Iterative development is so fundamental to software that I devote a complete chapter to it later in the book (Chapter 17). In this section, I introduce the topic by describing two process models that have been explicitly designed to support process iteration:

1. *Incremental delivery* The software specification, design and implementation are broken down into a series of increments that are each developed in turn.
2. *Spiral development* The development of the system spirals outwards from an initial outline through to the final developed system.

The essence of iterative processes is that the specification is developed in conjunction with the software. However, this conflicts with the procurement model of many organisations where the complete system specification is part of the system development contract. In the incremental approach, there is no complete system specification until the final increment is specified. This requires a new form of contract, which large customers such as government agencies may find difficult to accommodate.

4.2.1 Incremental delivery

The waterfall model of development requires customers for a system to commit to a set of requirements before design begins and the designer to commit to particular design strategies before implementation. Changes to the requirements require rework of the requirements, design and implementation. However, the separation of design and implementation should lead to well-documented systems that are amenable to change. By contrast, an evolutionary approach to development allows requirements and design decisions to be delayed but also leads to software that may be poorly structured and difficult to understand and maintain.

Incremental delivery (Figure 4.4) is an in-between approach that combines the advantages of these models. In an incremental development process, customers identify, in

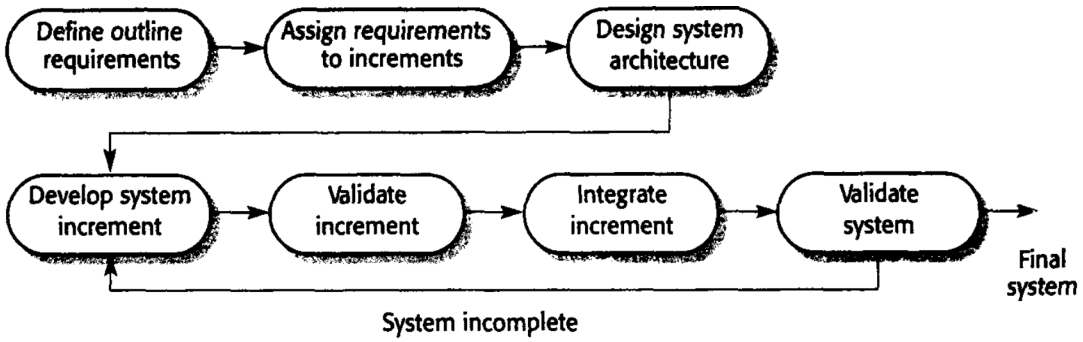


Figure 4.4 Incremental delivery

outline, the services to be provided by the system. They identify which of the services are most important and which are least important to them. A number of delivery increments are then defined, with each increment providing a sub-set of the system functionality. The allocation of services to increments depends on the service priority with the highest priority services delivered first.

Once the system increments have been identified, the requirements for the services to be delivered in the first increment are defined in detail, and that increment is developed. During development, further requirements analysis for later increments can take place, but requirements changes for the current increment are not accepted.

Once an increment is completed and delivered, customers can put it into service. This means that they take early delivery of part of the system functionality. They can experiment with the system that helps them clarify their requirements for later increments and for later versions of the current increment. As new increments are completed, they are integrated with existing increments so that the system functionality improves with each delivered increment. The common services may be implemented early in the process or may be implemented incrementally as functionality is required by an increment.

This incremental development process has a number of advantages:

1. Customers do not have to wait until the entire system is delivered before they can gain value from it. The first increment satisfies their most critical requirements so they can use the software immediately.
2. Customers can use the early increments as prototypes and gain experience that informs their requirements for later system increments.
3. There is a lower risk of overall project failure. Although problems may be encountered in some increments, it is likely that some will be successfully delivered to the customer.
4. As the highest priority services are delivered first, and later increments are integrated with them, it is inevitable that the most important system services receive the most testing. This means that customers are less likely to encounter software failures in the most important parts of the system.

However, there are problems with incremental delivery. Increments should be relatively small (no more than 20,000 lines of code), and each increment should deliver some system functionality. It can be difficult to map the customer s requirements onto increments of the right size. Furthermore, most systems require a set of basic facilities that are used by different parts of the system. As requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities that are needed by all increments.

A variant of this incremental approach called *extreme programming* has been developed (Beck, 2000). This is based around the development and delivery of very small increments of functionality, customer involvement in the process, constant code improvement and pair programming. I discuss extreme programming and other so-called agile methods in Chapter 17.

4.2.2 Spiral development

The spiral model of the software process (Figure 4.5) was originally proposed by Boehm (Boehm, 1988). Rather than represent the software process as a sequence of activities with some backtracking from one activity to another, the process is represented as a spiral. Each loop in the spiral represents a phase of the software process. Thus, the innermost loop might be concerned with system feasibility, the next loop with requirements definition, the next loop with system designand so on.

Each loop in the spiral is split into four sectors:

1. *Objective setting* Specific objectives for that phase of the project are defined. Constraints on the process and the product are identified and a detailed management plan is drawn up. Project risks are identified. Alternative strategies, depending on these risks, may be planned.

2. *Risk assessment and reduction* For each of the identified project risks, a detailed analysis is carried out. Steps are taken to reduce the risk. For example, if there is a risk that the requirements are inappropriate, a prototype system may be developed.

3. *Development and validation* After risk evaluation, a development model for the system is chosen. For example, if user interface risks are dominant, an appropriate development model might be evolutionary prototyping. If safety risks are the main consideration, development based on formal transformations may be the most appropriate and so on. The waterfall model may be the most appropriate development model if the main identified risk is sub-system integration.

4. *Planning* The project is reviewed and a decision made whether to continue with a further loop of the spiral. If it is decided to continue, plans are drawn up for the next phase of the project.

The main difference between the spiral model and other software process models is the explicit recognition of risk in the spiral model. Informally, risk simply means

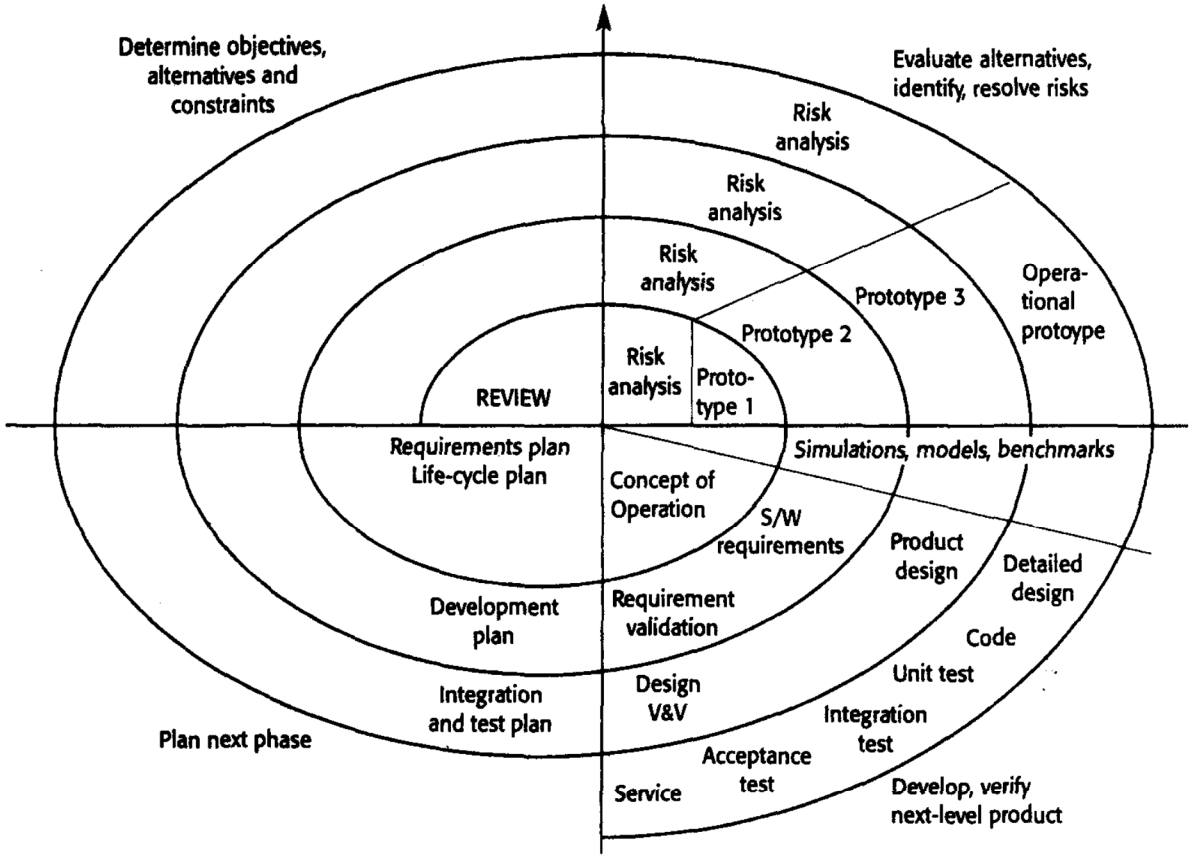


Figure 4.5 Boehm's spiral model of the software process (©IEEE, 1988)

something that can go wrong. For example, if the intention is to use a new programming language, a risk is that the available compilers are unreliable or do not produce sufficiently efficient object code. Risks result in project problems such as schedule and cost overrun so risk minimisation is a very important project management activity. Risk management, an essential part of project management, is covered in Chapter 5.

A cycle of the spiral begins by elaborating objectives such as performance and functionality. Alternative ways of achieving these objectives and the constraints imposed on each of them are then enumerated. Each alternative is assessed against each objective and sources of project risk are identified. The next step is to resolve these risks by information-gathering activities such as more detailed analysis, prototyping and simulation. Once risks have been assessed, some development is carried out, followed by a planning activity for the next phase of the process.

4.3 Process activities

The four basic process activities of specification, development, validation and evolution are organised differently in different development processes. In the waterfall

model, they are organised in sequence, whereas in evolutionary development they are interleaved. How these activities are carried out depends on the type of software, people and organisational structures involved. There is no right or wrong way to organise these activities and my goal in this section is simply to provide you with an introduction to how they can be organised.

4.3.1 Software specification

Software specification or requirements engineering is the process of understanding and defining what services are required from the system and identifying the constraints on the system's operation and development. Requirements engineering is a particularly critical stage of the software process as errors at this stage inevitably lead to later problems in the system design and implementation.

The requirements engineering process is shown in Figure 4.6. This process leads to the production of a requirements document that is the specification for the system. Requirements are usually presented at two levels of detail in this document. End-users and customers need a high-level statement of the requirements; system developers need a more detailed system specification.

There are four main phases in the requirements engineering process:

1. *Feasibility study* An estimate is made of whether the identified user needs may be satisfied using current software and hardware technologies. The study considers whether the proposed system will be cost-effective from a business point of view and whether it can be developed within existing budgetary constraints. A feasibility study should be relatively cheap and quick. The result should inform the decision of whether to go ahead with a more detailed analysis.

2. *Requirements elicitation and analysis* This is the process of deriving the system requirements through observation of existing systems, discussions with

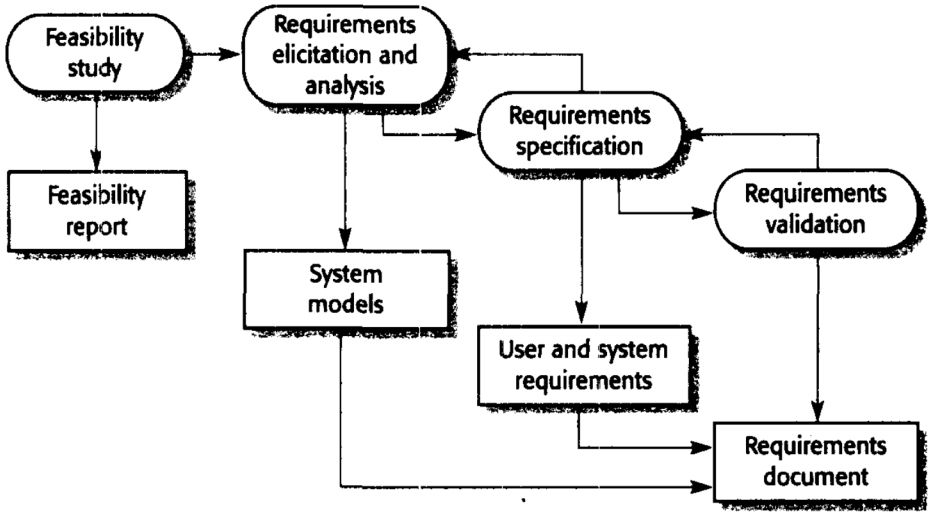


Figure 4.6 The requirements engineering process

potential users and procurers, task analysis and so on. This may involve the development of one or more system models and prototypes. These help the analyst understand the system to be specified.

3. *Requirements specification* The activity of translating the information gathered during the analysis activity into a document that defines a set of requirements. Two types of requirements may be included in this document. *User requirements* are abstract statements of the system requirements for the customer and end-user of the system; *system requirements* are a more detailed description of the functionality to be provided.

4. *Requirements validation* This activity checks the requirements for realism, consistency and completeness. During this process, errors in the requirements document are inevitably discovered. It must then be modified to correct these problems.

Of course, the activities in the requirements process are not simply carried out in a strict sequence. Requirements analysis continues during definition and specification, and new requirements come to light throughout the process. Therefore, the activities of analysis, definition and specification are interleaved. In agile methods such as extreme programming, requirements are developed incrementally according to user priorities, and the elicitation of requirements comes from users who are part of the development team.

4.3.2 Software design and implementation

The implementation stage of software development is the process of converting a system specification into an executable system. It always involves processes of software design and programming but, if an evolutionary approach to development is used, may also involve refinement of the software specification.

A software design is a description of the structure of the software to be implemented, the data which is part of the system, the interfaces between system components and, sometimes, the algorithms used. Designers do not arrive at a finished design immediately but develop the design iteratively through a number of versions. The design process involves adding formality and detail as the design is developed with constant backtracking to correct earlier designs.

The design process may involve developing several models of the system at different levels of abstraction. As a design is decomposed, errors and omissions in earlier stages are discovered. These feed back to allow earlier design models to be improved. Figure 4.7 is a model of this process showing the design descriptions that may be produced at various stages of design. This diagram suggests that the stages of the design process are sequential. In fact, design process activities are interleaved. Feedback from one stage to another and consequent design rework is inevitable in all design processes.

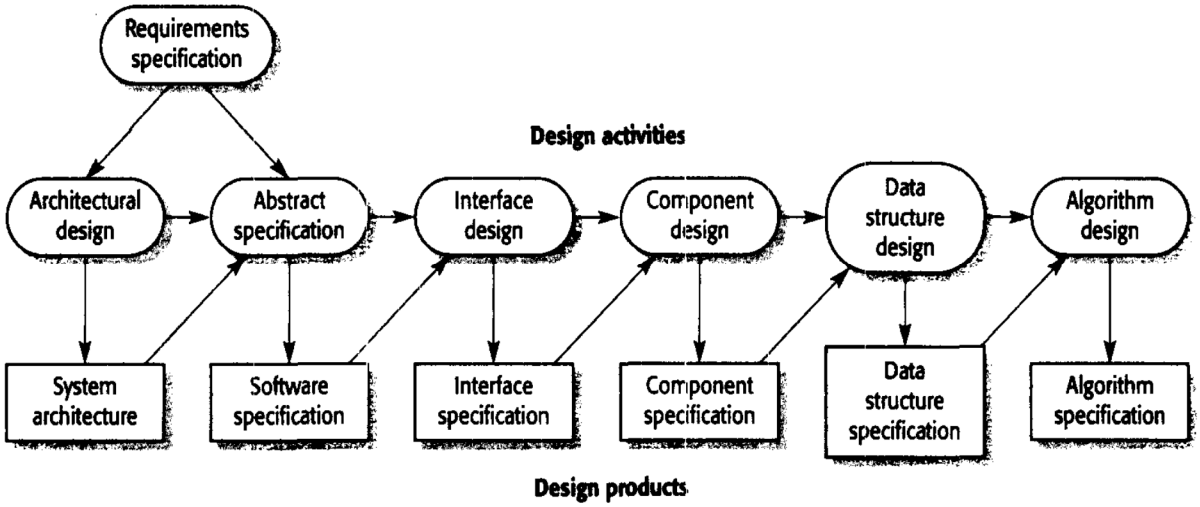


Figure 4.7 A general model of the design process

A specification for the next stage is the output of each design activity. This specification may be an abstract, formal specification that is prod ced to clarify the requirements, or it may be a specification of how part of the system is to be realised. As the design process continues, these specifications become more detailed. The final results of the process are precise specifications of the algorithms and data structures to be implemented.

The specific design process activities are:

1. *Architectural design* The sub-systems making up the system and their relationships are identified and documented. This important topic is covered in Chapters 11, 12 and 13.
2. *Abstract specification* For each sub-system, an abstract specification of its services and the constraints under which it must operate is produced.
3. *Interface design* For each sub-system, its interface with other sub-systems is designed and documented. This interface specification must be unambiguous as it allows the sub-system to be used without knowledge of the sub-system operation. Formal specification methods, as discussed in Chapter 10, may be used at this stage.
4. *Component design* Services are allocated to components and the interfaces of these components are designed.
5. *Data structure design* The data structures used in the system implementation are designed in detail and specified.
6. *Algorithm design* The algorithms used to provide services are designed in detail and specified.

This is a general model of the design process and real, practical processes may adapt it in different ways. Possible adaptations are:

1. The last two stages of design—data structure and algorithm design—may be delayed until the implementation process.
2. If an exploratory approach to design is used, the system interfaces may be designed after the data structures have been specified.
3. The abstract specification stage may be skipped, although it is usually an essential part of critical systems design.

Increasingly, where agile methods of development are used (see Chapter 17), the outputs of the design process will not be separate specification documents but will be represented in the code of the program. After the system architecture has been designed, later stages of the design are incremental. Each increment is represented as program code rather than as a design model.

A contrasting approach is taken by *structured methods* for design that rely on producing graphical models of the system (see Chapter 8) and, in many cases, automatically generating code from these models. Structured methods were invented in the 1970s to support function-oriented design (Constantine and Yourdon, 1979; Gane and Sarson, 1979). Various competing methods to support object-oriented design were proposed (Robinson, 1992; Booch, 1994) and these were unified in the 1990s to create the Unified Modeling Language (UML) and the associated unified design process (Rumbaugh, et al., 1991; Booch, et al., 1999; Rumbaugh, et al., 1999a; Rumbaugh, et al., 1999b). At the time of this writing, a major revision to UML (UML 2.0) is underway.

A structured method includes a design process model, notations to represent the design, report formats, rules and design guidelines. Structured methods may support some or all of the following models of a system:

1. An object model that shows the object classes used in the system and their dependencies.
2. A sequence model that shows how objects in the system interact when the system is executing.
3. A state transition model that shows system states and the triggers for the transitions from one state to another.
4. A structural model where the system components and their aggregations are documented.
5. A data flow model where the system is modelled using the data transformations that take place as it is processed. This is not normally used in object-oriented methods but is still frequently used in real-time and business system design.

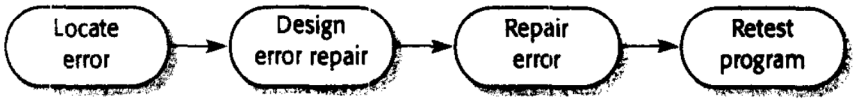


Figure 4.8 The debugging process

In practice, structured 'methods' are really standard notations and embodiments of good practice. Following these methods and applying the guidelines can result in a reasonable design. Designer creativity is still required to decide on the system decomposition and to ensure that the design adequately captures the system specification. Empirical studies of designers (Bansler and Bødker, 1993) have shown that they rarely follow methods slavishly. They pick and choose from the guidelines according to local circumstances.

The development of a program to implement the system follows naturally from the system design processes. Although some classes of programs, such as safety-critical systems, are usually designed in detail before any implementation begins, it is more common for the later stages of design and program development to be interleaved. CASE tools may be used to generate a skeleton program from a design. This includes code to define and implement interfaces, and in many cases the developer need only add details of the operation of each program component.

Programming is a personal activity and there is no general process that is usually followed. Some programmers start with components that they understand, develop them, and then move on to less well-understood components. Others take the opposite approach, leaving familiar components till last because they know how to develop them. Some developers like to define data early in the process then use this to drive the program development; others leave data unspecified for as long as possible.

Normally, programmers carry out some testing of the code they have developed. This often reveals program defects that must be removed from the program. This is called *debugging*. Defect testing and debugging are different processes. Testing establishes the existence of defects. Debugging is concerned with locating and correcting these defects.

Figure 4.8 illustrates the stages of debugging. Defects in the code must be located and the program modified to meet its requirements. Testing must then be repeated to ensure that the change has been made correctly. Thus the debugging process is part of both software development and software testing.

When debugging, you generate hypotheses about the observable behaviour of the program then test these hypotheses in the hope of finding the fault which caused the output anomaly. Testing the hypotheses may involve tracing the program code manually. You may write new test cases to localise the problem. Interactive debugging tools that show the intermediate values of program variables and a trace of the statements executed may be used to help the debugging process.

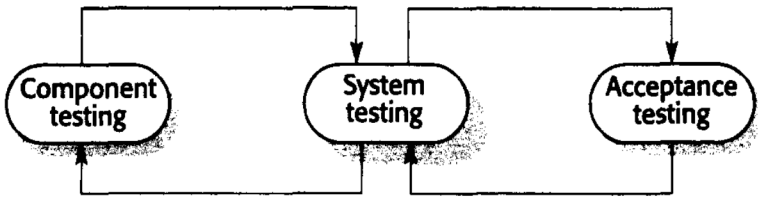


Figure 4.9 The testing process

4.3.3 Software validation

Software validation or, more generally, verification and validation (V & V) is intended to show that a system conforms to its specification and that the system meets the expectations of the customer buying the system. It involves checking processes, such as inspections and reviews (see Chapter 22), at each stage of the software process from user requirements definition to program development. The majority of validation costs, however, are incurred after implementation when the operational system is tested (Chapter 23).

Except for small programs, systems should not be tested as a single, monolithic unit. Figure 4.9 shows a three-stage testing process where system components are tested, the integrated system is tested and, finally, the system is tested with the customer's data. Ideally, component defects are discovered early in the process and interface problems when the system is integrated. However, as defects are discovered the program must be debugged and this may require other stages in the testing process to be repeated. Errors in program components, say, may come to light during system testing. The process is therefore an iterative one with information being fed back from later stages to earlier parts of the process.

The stages in the testing process are:

1. *Component (or unit) testing* Individual components are tested to ensure that they operate correctly. Each component is tested independently, without other system components. Components may be simple entities such as functions or object classes, or may be coherent groupings of these entities.
2. *System testing* The components are integrated to make up the system. This process is concerned with finding errors that result from unanticipated interactions between components and component interface problems. It is also concerned with validating that the system meets its functional and non-functional requirements and testing the emergent system properties. For large systems, this may be a multi-stage process where components are integrated to form sub-systems that are individually tested before they are themselves integrated to form the final system.
3. *Acceptance testing* This is the final stage in the testing process before the system is accepted for operational use. The system is tested with data supplied by the system customer rather than with simulated test data. Acceptance testing may reveal errors and omissions in the system requirements definition because the real data exercise the system in different ways from the test data. Acceptance testing may also reveal requirements problems where the system's facilities do not really meet the user s needs or the system performance is unacceptable.

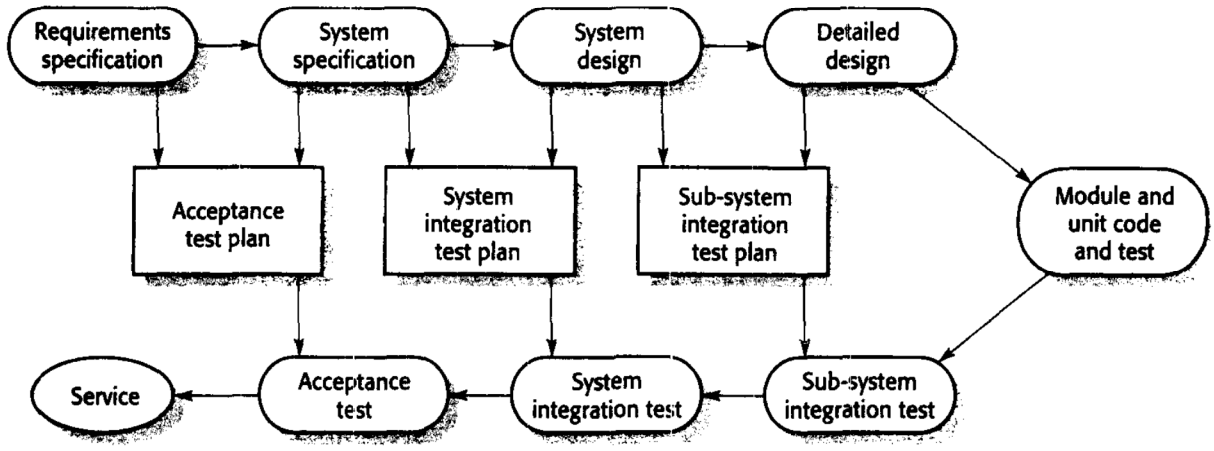


Figure 4.10 Testing phases in the software process

Normally, component development and testing are interleaved. Programmers make up their own test data and incrementally test the code as it is developed. This is an economically sensible approach, as the programmer knows the component best and is therefore the best person to generate test cases.

If an incremental approach to development is used, each increment should be tested as it is developed, with these tests based on the requirements for that increment. In extreme programming, tests are developed along with the requirements before development starts. This helps the testers and developers to understand the requirements and ensures that there are no delays as test cases are created.

Later stages of testing involve integrating work from a number of programmers and must be planned in advance. An independent team of testers should work from preformulated test plans that are developed from the system specification and design. Figure 4.10 illustrates how test plans are the link between testing and development activities.

Acceptance testing is sometimes called alpha testing. Custom systems are developed for a single client. The alpha testing process continues until the system developer and the client agree that the delivered system is an acceptable implementation of the system requirements.

When a system is to be marketed as a software product, a testing process called beta testing is often used. Beta testing involves delivering a system to a number of potential customers who agree to use that system. They report problems to the system developers. This exposes the product to real use and detects errors that may not have been anticipated by the system builders. After this feedback, the system is modified and released either for further beta testing or for general sale.

4.3.4 Software evolution

The flexibility of software systems is one of the main reasons why more and more software is being incorporated in large, complex systems. Once a decision has been made to procure hardware, it is very expensive to make changes to the hardware

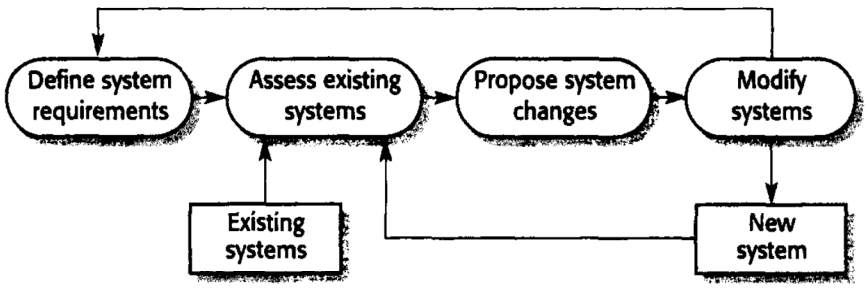


Figure 4.11 System evolution

design. However, changes can be made to software at any time during or after the system development. Even extensive changes are still much cheaper than corresponding changes to system hardware.

Historically, there has always been a split between the process of software development and the process of software evolution (software maintenance). People think of software development as a creative activity where a software system was developed from an initial concept through to a working system. However, they sometimes think of software maintenance as dull and uninteresting. Although the costs of 'maintenance are often several times the initial development costs, maintenance processes are sometimes considered to be less challenging than original software development.

This distinction between development and maintenance is becoming increasingly irrelevant. Few software systems are now completely new systems, and it makes much more sense to see development and maintenance as a continuum. Rather than two separate processes, it is more realistic to think of software engineering as an evolutionary process (Figure 4.11) where software is continually changed over its lifetime in response to changing requirements and customer needs.

4.4 The Rational Unified Process

The Rational Unified Process (RUP) is an example of a modern process model that has been derived from work on the UML and the associated Unified Software Development Process (Rumbaugh, et al., 1999b). I have included a description here as it is a good example of a hybrid process model. It brings together elements from all of the generic process models (Section 4.1), supports iteration (Section 4.2) and illustrates good practice in specification and design (Section 4.3).

The RUP recognises that conventional process models present a single view of the process. In contrast, the RUP is normally described from three perspectives:

1. A dynamic perspective that shows the phases of the model over time.
2. A static perspective that shows the process activities that are enacted.
3. A practice perspective that suggests good practices to be used during the process.

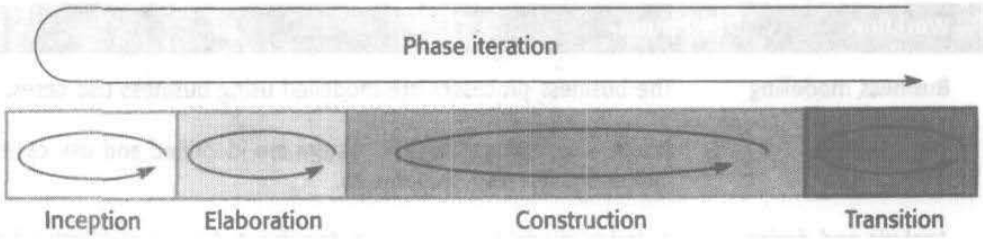


Figure 4.12 Phases in the Rational Unified Process

Most descriptions of the RUP attempt to combine the static and dynamic perspectives in a single diagram (Krutchen, 2000). I think that makes the process harder to understand, so I use separate descriptions of each of these perspectives.

The RUP is a phased model that identifies four discrete phases in the software process. However, unlike the waterfall model where phases are equated with process activities, the phases in the RUP are more closely related to business rather than technical concerns. Figure 4.12 shows the phases in the RUP. These are:

1. *Inception* The goal of the inception phase is to establish a business case for the system. You should identify all external entities (people and systems) that will interact with the system and define these interactions. You then use this information to assess the contribution that the system makes to the business. If this contribution is minor, then the project may be cancelled after this phase.
2. *Elaboration* The goals of the elaboration phase are to develop an understanding of the problem domain, establish an architectural framework for the system, develop the project plan and identify key project risks. On completion of this phase, you should have a requirements model for the system (UML use cases are specified), an architectural description and a development plan for the software.
3. *Construction* The construction phase is essentially concerned with system design, programming and testing. Parts of the system are developed in parallel and integrated during this phase. On completion of this phase, you should have a working software system and associated documentation that is ready for delivery to users.
4. *Transition* The final phase of the RUP is concerned with moving the system from the development community to the user community and making it work in a real environment. This is something that is ignored in most software process models but is, in fact, an expensive and sometimes problematic activity. On completion of this phase, you should have a documented software system that is working correctly in its operational environment.

Iteration within the RUP is supported in two ways, as shown in Figure 4.12. Each phase may be enacted in an iterative way with the results developed incrementally. In addition, the whole set of phases may also be enacted incrementally, as shown by the looping arrow from Transition to Inception in Figure 4.12.

Workflow	Description
Business modelling	The business processes are modelled using business use cases.
Requirements	Actors who interact with the system are identified and use cases are developed to model the system requirements.
Analysis and design	A design model is created and documented using architectural models, component models, object models and sequence models.
Implementation	The components in the system are implemented and structured into implementation sub-systems. Automatic code generation from design models helps accelerate this process.
Testing	Testing is an iterative process that is carried out in conjunction with implementation. System testing follows the completion of the implementation.
Deployment	A product release is created, distributed to users and installed in their workplace.
Configuration and change management	This supporting workflow manages changes to the system (see Chapter 29).
Project management	This supporting workflow manages the system development (see Chapter 5).
Environment	This workflow is concerned with making appropriate software tools available to the software development team.

Figure 4.13 Static workflows in Rational Unified Process

The static view of the RUP focuses on the activities that take place during the development process. These are called *workflows* in the RUP description. There are six core process workflows identified in the process and three core supporting workflows. The RUP has been designed in conjunction with the UML—an object-oriented modelling language—so the workflow description is oriented around associated UML models. The core engineering and support workflows are described in Figure 4.13.

The advantage in presenting dynamic and static views is that phases of the development process are not associated with specific workflows. In principle at least, all of the RUP workflows may be active at all stages of the process. Of course, most effort will probably be spent on workflows such as business modelling and requirements at the early phases of the process and in testing and deployment in the later phases.

The practice perspective on the RUP describes good software engineering practices that are recommended for use in systems development. Six fundamental best practices are recommended:

1. *Develop software iteratively*. Plan increments of the system based on customer priorities and develop and deliver the highest priority system features early in the development process.
2. *Manage requirements*. Explicitly document the customer s requirements and keep track of changes to these requirements. Analyse the impact of changes on the system before accepting them.

3. *Use component-based architectures.* Structure the system architecture into components as discussed earlier in this chapter.
4. *Visually model software.* Use graphical UML models to present static and dynamic views of the software.
5. *Verify software quality.* Ensure that the software meets the organisational quality standard .
6. *Control changes to software.* Manage changes to the software using a change management system and configuration management procedures and tools (see Chapter 29).

The RUP is not a suitable process for all types of development but it does represent a new generation of generic processes. The most important innovations are the separation of phases and workflows, and the recognition that deploying software in a user s environment is part of the process. Phases are dynamic and have goals. Workflows are static and are technical activities that are not associated with a single phase but may be used throughout the development to achieve the goals of each phase.

4.5 Computer-Aided Software Engineering

Computer-Aided Software Engineering (CASE) is the name given to software used to support software process activities such as requirements engineering, design, program development and testing. CASE tools therefore include design editors, data dictionaries, compilers, debuggers, system building tools and so on.

CASE technology provides software process support by automating some process activities and by providing information about the software that is being developed. Examples of activities that can be automated using CASE include:

1. The development of graphical system models as part of the requirements specification or the software design.
2. Understanding a design using a data dictionary that holds information about the entities and relations in a design.
3. The generation of user interfaces from a graphical interface description that is created interactively by the user.
4. Program debugging through the provision of information about an executing program.
5. The automated translation of programs from an old version of a programming language such as COBOL to a more recent version.

CASE technology is now available for most routine activities in the software process. This has led to some improvements in software quality and productivity, although these have been less than predicted by early advocates of CASE. Early advocates suggested that orders of magnitude improvement were likely if integrated CASE environments were used. In fact, the improvements that have been achieved are of the order of 40% (Huff, 1992). Although this is significant, the predictions when CASE tools were first introduced in the 1980s and 1990s were that the use of CASE technology would generate huge savings in software process costs.

The improvements from the use of CASE are limited by two factors:

1. Software engineering is, essentially, a design activity based on creative thought. Existing CASE systems automate routine activities but attempts to harness artificial intelligence technology to provide support for design have not been successful.
2. In most organisations, software engineering is a team activity, and software engineers spend quite a lot of time interacting with other team members. CASE technology does not provide much support for this.

CASE technology is now mature, and CASE tools and workbenches are available from a wide range of suppliers. However, rather than focus on any specific tools, I simply present an overview of tools here with some discussion of specific support in other chapters. In my web pages, I include links to other material on CASE and links to CASE tool suppliers.

4.5.1 CASE classification

CASE classifications help us understand the types of CASE tools and their role in supporting software process activities. There are several ways to classify CASE tools, each of which gives us a different perspective on these tools. In this section, I discuss CASE tools from three of these perspectives:

1. *A functional perspective* where CASE tools are classified according to their specific function.
2. *A process perspective* where tools are classified according to the process activities that they support.
3. *An integration perspective* where CASE tools are classified according to how they are organised into integrated units that provide support for one or more process activities.

Figure 4.14 is a classification of CASE tools according to function. This table lists a number of different types of CASE tools and gives specific examples of each

Figure 4.14 Functional classification of CASE tools

Tool type	Examples
Planning tools	PERT tools, estimation tools, spreadsheets
Editing tools	Text editors, diagram editors, word processors
Change management tools	Requirements traceability tools, change control systems
Configuration management tools	Version management systems, system building tools
Prototyping tools	Very high-level languages, user interface generators
Method-support tools	Design editors, data dictionaries, code generators
Language-processing tools	Compilers, interpreters
Program analysis tools	Cross reference generators, static analysers, dynamic analysers
Testing tools	Test data generators, file comparators
Debugging tools	Interactive debugging systems
Documentation tools	Page layout programs, image editors
Reengineering tools	Cross-reference systems, program restructuring systems

one. This is not a complete list of CASE tools. Specialised tools, such as tools to support reuse, have not been included.

Figure 4.15 presents an alternative classification of CASE tools. It shows the process phases supported by a number of types of CASE tools. Tools for planning and estimating, text editing, document preparation and configuration management may be used throughout the software process.

The breadth of support for the software process offered by CASE technology is another possible classification dimension. Fuggetta (Fuggetta, 1993) proposes that CASE systems should be classified in three categories:

1. *Tools* support individual process tasks such as checking the consistency of a design, compiling a program and comparing test results. Tools may be general-purpose, standalone tools (e.g., a word processor) or grouped into workbenches.
2. *Workbenches* support process phases or activities such as specification, design, etc. They normally consist of a set of tools with some greater or lesser degree of integration.
3. *Environments* support all or at least a substantial part of the software process. They normally include several integrated workbenches.

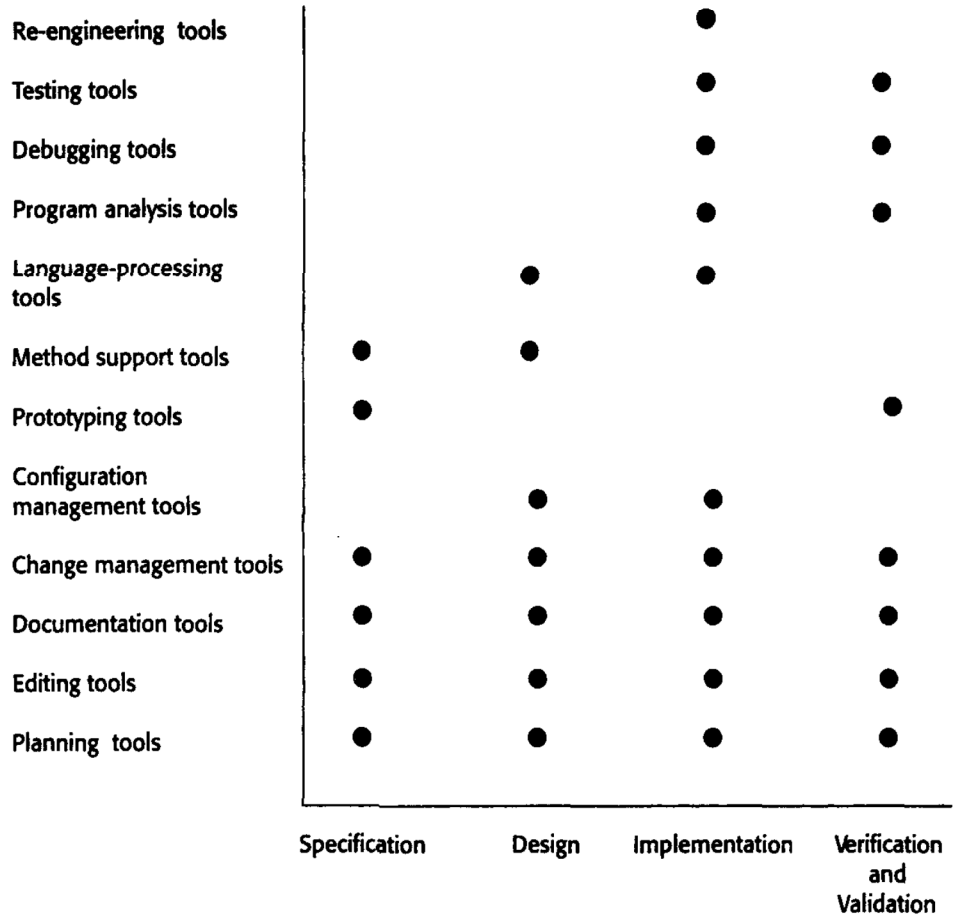


Figure 4.15 Activity-based classification of CASE tools

Figure 4.16 illustrates this classification and shows some examples of these classes of CASE support. Of course, this is an illustrative example; many types of tools and workbenches have been left out of this diagram.

General-purpose tools are used at the discretion of the software engineer who makes decisions about when to apply them for process support. Workbenches, however, usually support some method that includes a process model and a set of rules/guidelines, which apply to the software being developed. I have classified environments as integrated or process-centred. Integrated environments provide infrastructure support for data, control and presentation integration. Process-centred environments are more general. They include software process knowledge and a process engine which uses this process model to advise engineers on what tools or workbenches to apply and when they should be used.

In practice, the boundaries between these classes are blurred. Tools may be sold as a single product but may embed support for different activities. For example, most word processors now provide a built-in diagram editor. CASE workbenches for design usually support programming and testing, so they are more akin to environments than specialised workbenches. It may therefore not always be easy to position a product using a classification. Nevertheless, classification provides a useful first step to help understand the extent of process support that a tool provides.

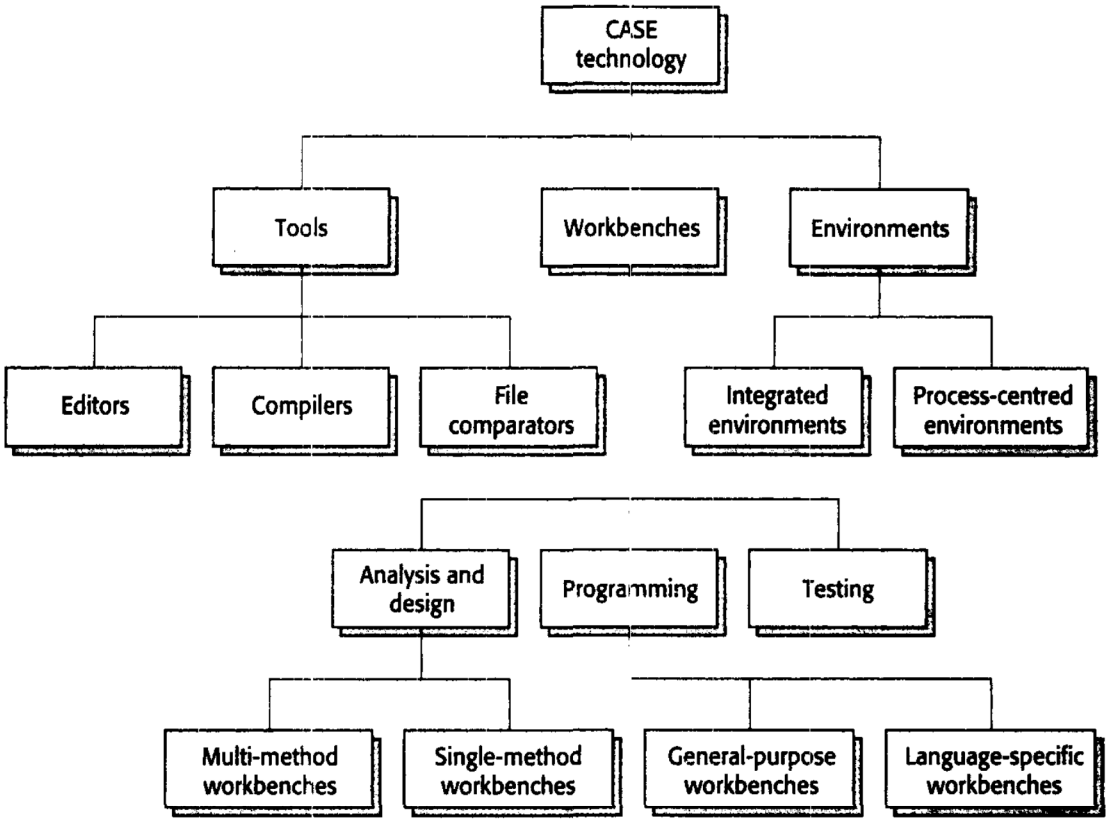


Figure 4.16 Tools, workbenches and environments



KEY POINTS

Software processes are the activities involved in producing a software system. Software process models are abstract representations of these processes.

All software processes include software specification, software design and implementation, software validation and software evolution.

Generic process models describe the organisation of software processes. Examples of generic models include the waterfall model, evolutionary development and component-based software engineering.

Iterative process models present the software process as a cycle of activities. The advantage of this approach is that it avoids premature commitments to a specification or design.

Examples of iterative models include incremental development and the spiral model.

Requirements engineering is the process of developing a software specification. Specifications are intended to communicate the system needs of the customer to the system developers.

Design and implementation processes are concerned with transforming a requirements specification into an executable software system. Systematic design methods may be used as part of this transformation.

Software validation is the process of checking that the system conforms to its specification and that it meets the real needs of the users of the system.

Software evolution is concerned with modifying existing software systems to meet new requirements. This is becoming the normal approach to software development for small and medium-sized systems.

The Rational Unified Process is a modern generic process model that is organised into phases (inception, elaboration, construction and transition) but that separates activities (requirements, analysis and design, etc.) from these phases.

CASE technology provides automated support for software processes. CASE tools support individual process activities; workbenches support a set of related activities; environments support all or most software process activities.

FURTHER READING

Extreme Programming Explained: Embrace Change. An evangelical book that describes the extreme programming process and extreme programming experiences. The author was the inventor of extreme programming and communicates his enthusiasm very well. (Kent Beck, 2000, Addison-Wesley.)

The Rational Unified Process—An Introduction. This is the most readable book available on the RUP at the time of this writing. Krutchen describes the process well, but I would like to have seen more on the practical difficulties of using the process. (P. Krutchen, 2000, Addison-Wesley.)

Managing Software Quality and Business Risk. This is primarily a book about software management but it includes an excellent chapter (Chapter 4) on process models. (M. Ould, 1999, John Wiley & Sons)

'A classification of CASE technology'. The classification scheme proposed in this article is used in this chapter, but Fuggetta goes into more detail and illustrates how a number of commercial products fit into this scheme. (A. Fuggetta, *IEEE Computer*, 26 (12), December 1993.)

EXERCISES

4.1 Giving reasons for your answer based on the type of system being developed, suggest the most appropriate generic software process model that might be used as a basis for managing the development of the following systems:

- A system to control anti-lock braking in a car
- A virtual reality system to support software maintenance
- A university accounting system that replaces an existing system
- An interactive system that allows railway passengers to find train times from terminals installed in stations.

4.2 Explain why programs that are developed using evolutionary development are likely to be difficult to maintain.

4.3 Explain how both the waterfall model of the software process and the prototyping model can be accommodated in the spiral process model.

4.4 What are the advantages of providing static and dynamic views of the software process as in the Rational Unified Process?

4.5 Suggest why it is important to make a distinction between developing the user requirements and developing system requirements in the requirements engineering process.

4.6 Describe the main activities in the software design process and the outputs of these activities. Using a diagram, show possible relationships between the outputs of these activities.

4.7 What are the five components of a design method? Take any method you know and describe its components. Assess the completeness of the method that you have chosen.

4.8 Design a process model for running system tests and recording their results.

4.9 Explain why a software system that is used in a real-world environment must change or become progressively less useful.

4.10 Suggest how a CASE technology classification scheme may be helpful to managers responsible for CASE system procurement.

4.11 Survey the tool availability in your local development environment and classify the tools according to the parameters (function, activity, breadth of support) suggested here.

4.12 Historically, the introduction of technology has caused profound changes in the labour market and, temporarily at least, displaced people from jobs. Discuss whether the introduction of advanced CASE technology is likely to have the same consequences for software engineers. If you don't think it will, explain why not. If you think that it will reduce job opportunities, is it ethical for the engineers affected to passively or actively resist the introduction of this technology?